Monte Carlo Options Pricing System

M4 — Options Pricing — Design Checkpoint

Yassine

August 2026

1 Milestone Status

Milestone	Status	Tests
M1 — Data Ingestion	Complete	24
M2 — Calibration Engine	Complete	24
M3 — Simulation Engine	Complete	72
M4 — Options Pricing	Complete	87
Total		**183**

All 96 prior tests (M2 + M3) unchanged.

2 Risk-Neutral Pricing — Core Concept

Under the physical measure, simulations use the historical drift μ. For pricing derivatives, we simulate under the *risk-neutral measure*, replacing μ with $r - q$ (risk-free rate minus dividend yield):

$$\text{Option Price} = e^{-rT} \cdot \frac{1}{N} \sum_{i=1}^{N} \text{Payoff}(S^{(i)})$$

The pricer passes `mu = r - q` as a keyword argument override to all simulators. Each simulator applies it differently:

- **GBM**: drift $= r - q$ directly.
- **Jump-Diffusion**: effective drift $= (r - q) - \lambda_j k$, where $k = e^{\mu_j + \sigma_j^2/2} - 1$ is the jump compensator. The simulator already subtracts $\lambda_j k$ internally.
- **Heston**: `mu` kwarg already existed from M3 design; pricer passes $r - q$.

3 File Structure

```
src/pricing/
  __init__.py
  payoffs.py          # 8 payoff callables + PAYOFF_REGISTRY
  pricer.py           # MCPricer + PricingResult
  black_scholes.py    # bs_call, bs_put, bs_greeks
  greeks.py           # GreeksCalculator + GreeksResult
```

4 Architecture Decisions

4.1 Payoffs — Simple Callables

Every payoff function follows a uniform signature:

```
def payoff_fn(paths: np.ndarray, K: float, **kwargs) -> np.ndarray:
    # paths: (n_sims, n_steps+1), returns: (n_sims,)
```

A flat dictionary maps string names to callables:

```
PAYOFF_REGISTRY = {
    "european_call": european_call,
    "european_put":  european_put,
    ...
}
```

The pricer passes `**kwargs` through, so barrier parameters, etc. flow naturally without metadata in the registry.

Implemented payoffs (8 total):

Payoff	Formula	Notes
European call	$\max(S_T - K, 0)$	Terminal only
European put	$\max(K - S_T, 0)$	Terminal only
Asian call	$\max(\bar{S} - K, 0)$	$\bar{S}$ excludes S_0
Asian put	$\max(K - \bar{S}, 0)$	$\bar{S}$ excludes S_0
Barrier KO call	European call if no breach	Up-and-out ($S \geq B$)
Barrier KO put	European put if no breach	Down-and-out ($S \leq B$)
Lookback call	$S_T - \min(S_{1:})$	Floating strike, no K
Lookback put	$\max(S_{1:}) - S_T$	Floating strike, no K

4.2 MCPricer — Dependency Injection

Receives a `SimulatorBase` instance via constructor injection. Never imports a specific simulator.

```
class MCPricer:
    def __init__(self, simulator: SimulatorBase):
        self.simulator = simulator

    def price(self, option_type, S0, K, T, r, model_params,
              n_simulations=50000, dt=1/252, q=0.0,
              seed=None, variance_reduction=None,
              **payoff_kwargs) -> PricingResult
```

Internal flow: registry lookup → simulate with `mu=r-q` → compute payoffs → discount at e^{-rT} → standard error → CI → optional BS benchmark.

4.3 PricingResult — Lean Output

```
class PricingResult(BaseModel):
    price: float
    std_error: float
    confidence_interval_95: tuple[float, float]
    n_simulations: int
    computation_time_ms: float
    bs_benchmark: float | None = None
    bs_relative_error: float | None = None
```

No raw payoff arrays stored. BS benchmark computed automatically for European options when params have a `sigma` attribute (GBM, JD). For JD, BS serves as an inequality reference (MC > BS due to jumps), not a convergence target.

4.4 Black-Scholes — Standalone Functions

Pure functions with dividend yield support:

$$C = S_0 e^{-qT} N(d_1) - K e^{-rT} N(d_2)$$

$$d_1 = \frac{\ln(S_0/K) + (r - q + \sigma^2/2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}$$

`bs_greeks()` returns analytical Delta, Gamma, Vega, Theta, Rho — used as benchmark for MC Greeks.

4.5 Greeks — Separate Calculator, Common Random Numbers

`GreeksCalculator` receives `MCPricer` via DI. The `seed` parameter is *required* (not optional) — common random numbers are fundamental to bump-and-revalue.

Greek	Input bumped	Bump size	Method	Notes
Δ	S_0	$\pm 1\%$ of S_0	Central diff	
Γ	S_0	$\pm 1\%$ of S_0	2nd-order central	
$\mathcal{V}$	σ (or v_0)	± 0.01 absolute	Central diff	Heston: bumps $\sqrt{v_0}$
Θ	T	$-1/252$	Forward diff	Holds n_{steps} fixed
ρ	r	± 10 bps	Central diff	

5 M3 Simulator Changes for M4

Added `mu: float | None = None` to `simulate()` on GBM and JD simulators. When provided, overrides `params.mu`. Defaults to `None` for backwards compatibility — all M3 tests pass unchanged.

6 Bugs Found and Fixed

6.1 Theta — Broken Common Random Numbers

Problem: Bumping T changed `n_steps = round(T/dt)`, altering the random draw shape. Base and bumped runs used effectively independent samples despite sharing a seed. Result: Theta was pure noise.

Why it was hidden: At $T = 1.0$, noise happened to land near the correct value. The spec's single-seed test passed by coincidence.

Fix: `_matching_dt()` holds `n_steps` fixed and adjusts `dt` for the bumped run. Draws stay bit-identical; only time spacing changes. Theta now matches BS analytical: -10.470 (MC) vs -10.474 (BS). Regression test parametrizes over three seeds.

6.2 T = 1/252 Crash

Problem: Theta guard `T >= 1/252` allowed a bumped horizon of exactly 0, which `resolve_n_steps` rejects.

Fix: Guard changed to `T >= dt`.

7 Deliberate Deviations from Spec

7.1 Antithetic Standard Error Estimator

The spec prescribed SE = std(payoffs, ddof = 1)/$\sqrt{n}$, which assumes i.i.d. draws. Antithetic variates break i.i.d. by design — path i and its antithetic partner are negatively correlated. Measured overstating factor: 1.8×.

Implemented: pair-average estimator. Average each (path, antithetic) pair first, then compute variance on the $n/2$ averages. Calibrated ratio: 0.94 (vs 0.96 for naive i.i.d. on plain MC).

7.2 Stratified Standard Error

Kept the conservative i.i.d. formula — one-draw-per-stratum admits no simple unbiased variance estimator. CI is $\sim 2\times$ wider than ideal. Tightening requires replicated stratification with batched variance estimation. Deferred.

8 Validation Results

Textbook parameters: $S_0 = 100$, $K = 100$, $T = 1.0$, $r = 0.05$, $\sigma = 0.2$, $q = 0.0$.

Metric	MC (antithetic)	Analytic (BS)	Relative Error
European call price	10.4297	10.4506	0.20%
Δ	—	—	0.09%
Γ	—	—	1.7%
$\mathcal{V}$	—	—	0.02%
Θ	—	—	0.5%
ρ	—	—	0.14%

9 Design Invariants Maintained

- Pricer decoupled from simulators via DI — never imports a specific simulator
- Greeks calculator decoupled from pricer via DI
- Payoffs are simple functions, not classes
- No MLflow logging inside pricing code — caller's responsibility
- `ddof=1` for standard error computation
- Barriers and lookbacks monitored discretely at step dates (not continuously)
- `bs_relative_error` returns `None` when benchmark ≤ 0 (deep OTM)

10 Next: M5 — Risk Analytics

VaR, CVaR, max drawdown, Sharpe ratio, probability of loss. Multi-asset correlated simulation via Cholesky decomposition. Stress testing with scenario overrides.